T3 Firmware & Bootloader

Resuming an interrupted update · built Nov 13, 2026

Resuming an interrupted update

A transfer over RS485 to a controller in a plant room will be interrupted. The bootloader records how far it got, so the next attempt continues from there instead of starting again — and there is an erase path when resuming is not what you want.

5. Resumable Firmware Update Mechanism

Packet Structure

TEXT

```
Each firmware packet is 128 bytes:

Packet Format:
┌──────────────────────────────────────┐
│ Packet Header (varies by protocol) │ ← Protocol dependent
├──────────────────────────────────────┤
│ Firmware Data (up to 128 bytes)    │ ← Actual firmware code/data
└──────────────────────────────────────┘

OTA Packet Addressing:
- Packet 0-N numbered sequentially
- Each packet = 128 bytes
- Flash offset = Packet_Number × 128
- Example: Packet 5 starts at byte offset 640 (5 × 128)
```

Resume State Tracking

TEXT

```
Flash Storage (NVS):

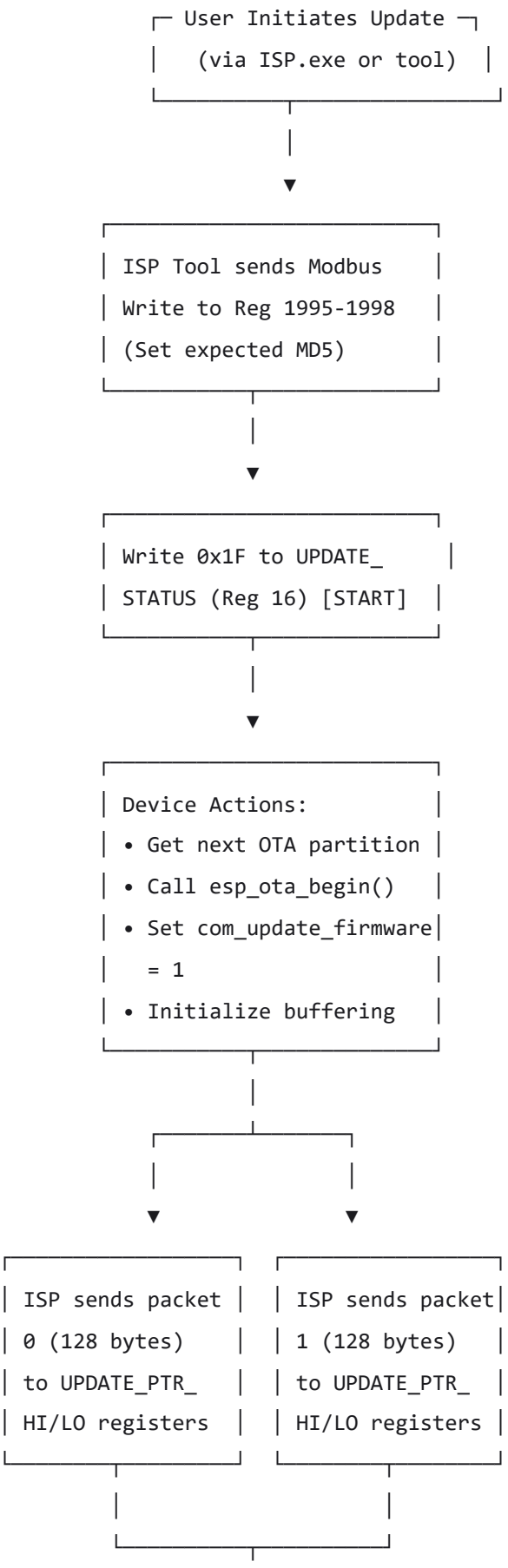
┌──────────────────────────────────────────────────────────┐
│ Key: "RS_RES_PACKAGE"                                    │
│ Value: uint32_t rs485_resume_chunk (packet count)        │
│                                                          │
│ On boot, this value is loaded into:                      │
│   rs485_resume_chunk = load_uint32_from_flash(...)       │
│                                                          │
│ Example: rs485_resume_chunk = 100 means:                 │
│   ├─ Update successfully wrote packets 0-99              │
│   ├─ Next update should start at packet 100              │
│   ├─ Flash byte offset = 100 × 128 = 12,800 bytes        │
│   └─ UPDATE_PTR_HI = 0x50, UPDATE_PTR_LO = 0x00          │
└──────────────────────────────────────────────────────────┘

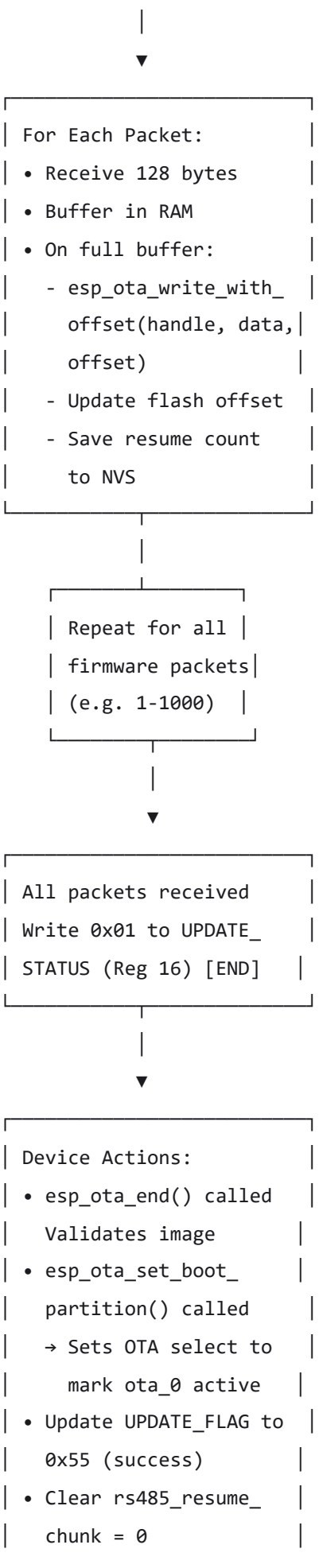

Runtime Variables:

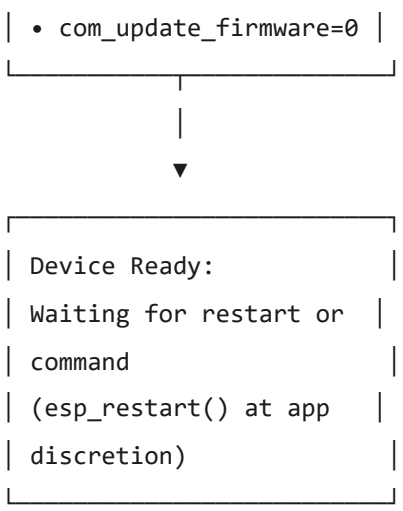
┌──────────────────────────────────────────────────────────┐
│ rs485_resume_chunk                                       │
│  └─ Packet count (0 to max packets)                      │
│                                                          │
│ com_flash_high / com_flash_low                           │
│  ├─ Derived from packet count                            │
│  └─ Used for protocol feedback to ISP tool               │
│                                                          │
│ s_rs485_ota_next_offset                                  │
│  ├─ Byte offset in flash = resume_chunk × 128            │
│  └─ esp_ota_write_with_offset() writes starting here     │
│                                                          │
│ s_rs485_ota_chunk_len / s_rs485_ota_chunk_buf[]          │
│  ├─ Buffering: accumulate received data                  │
│  ├─ When buffer full (128 bytes), flush to flash         │
│  └─ Ensures atomic flash writes                          │
└──────────────────────────────────────────────────────────┘
```

6. Complete Firmware Update Flow - Normal Path

TEXT

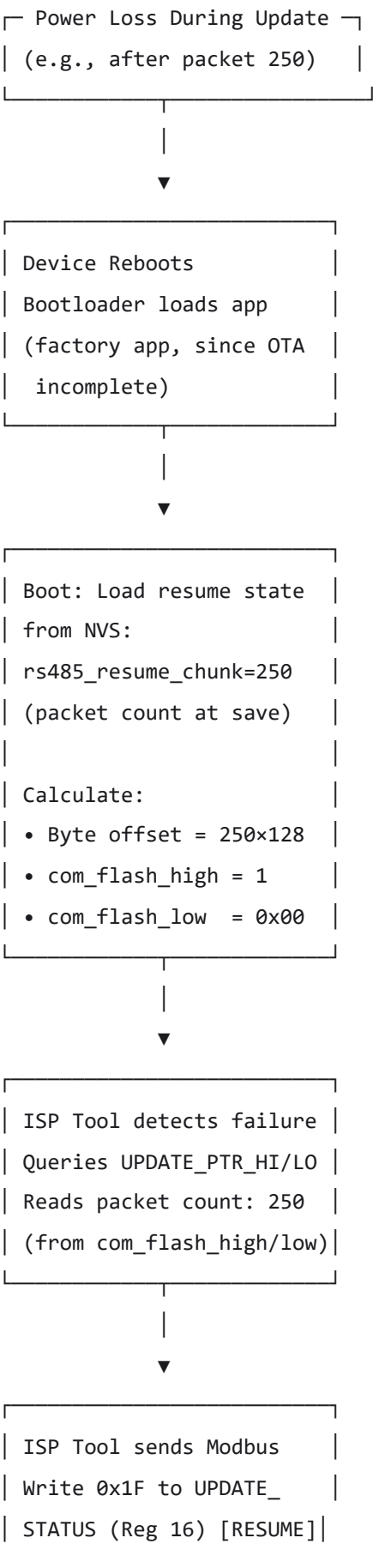


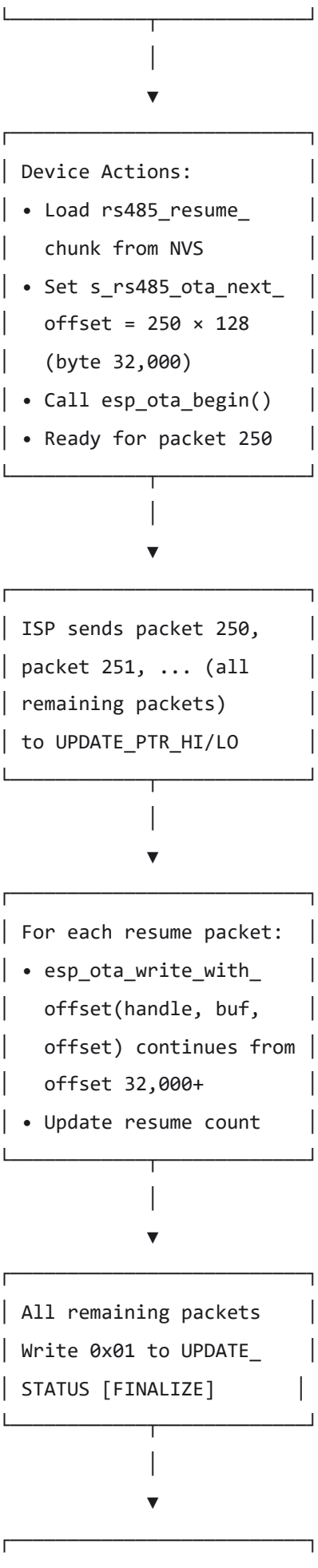

For Each Packet:
• Receive 128 bytes
• Buffer in RAM
• On full buffer:
- esp_ota_write_with_offset(handle, data, offset)
- Update flash offset
- Save resume count to NVS
Repeat for all firmware packets (e.g. 1-1000)
All packets received
Write 0x01 to UPDATE_STATUS (Reg 16) [END]
Device Actions:
• esp_ota_end() called
Validates image
• esp_ota_set_boot_partition() called
→ Sets OTA select to mark ota_0 active
• Update UPDATE_FLAG to 0x55 (success)
• Clear rs485_resume_chunk = 0


• com_update_firmware=0
Device Ready:
Waiting for restart or
command
(esp_restart() at app
discretion)

7. Resumable Firmware Update Flow - Recovery Path

TEXT




Device Actions:
• Load rs485_resume_ chunk from NVS
• Set s_rs485_ota_next_ offset = 250 × 128 (byte 32,000)
• Call esp_ota_begin()
• Ready for packet 250
ISP sends packet 250, packet 251, ... (all remaining packets) to UPDATE_PTR_HI/LO
For each resume packet:
• esp_ota_write_with_ offset(handle, buf, offset) continues from offset 32,000+
• Update resume count
All remaining packets Write 0x01 to UPDATE_ STATUS [FINALIZE]

```
│ • esp_ota_end()          │
│ • esp_ota_set_boot_      │
│   partition()            │
│ • UPDATE_FLAG = 0x55     │
│ • Ready to boot new app  │
└──────────────────────────┘
```

8. Erase Operation (Full Reset)

TEXT

```
┌─ ISP Tool Error Detection ─┐
│ (e.g. wrong image)         │
└──────────────┬─────────────┘
               │
               ▼
┌──────────────────────────┐
│ ISP sends Modbus command │
│ Write 0x3F to UPDATE_    │
│ STATUS (Reg 16) [ERASE]  │
└──────────────┬───────────┘
               │
               ▼
┌──────────────────────────┐
│ Device Actions:          │
│ • Abort any in-progress  │
│   esp_ota_abort()        │
│ • esp_partition_erase_   │
│   range() on full OTA    │
│ • Clear NVS resume data: │
│   rs485_resume_chunk=0   │
│ • Reset all buffers:     │
│   s_rs485_ota_chunk_len  │
│   = 0                    │
│ • com_update_firmware=0  │
│ • Ready for fresh update │
└──────────────────────────┘
```